Department of Physics and Astronomy

Techniques of High-Performance Computing - PHAS0102

Week 1: Numpy + Parallelism

HPC Programming Languages

- **Compiled Languages**
  - C/C++
  - Rust
  - Fortran
  - Go
  - COBOL
  - Haskell
  - Pascal
- **Interpreted Languages**
  - Python
  - Ruby
  - Matlab
  - Javascript
  - Lua
  - Maple
  - Mathematica

Python

- Introduced in 1980 by Guido van Rossum
- Dynamically Typed
- Focuses on flexibility and readability
- Indentation rather than braces
- Actually a standard!
  - ‘Python’ is actually CPython
  - PyPy
  - Jython
  - IronPython

Zen of Python

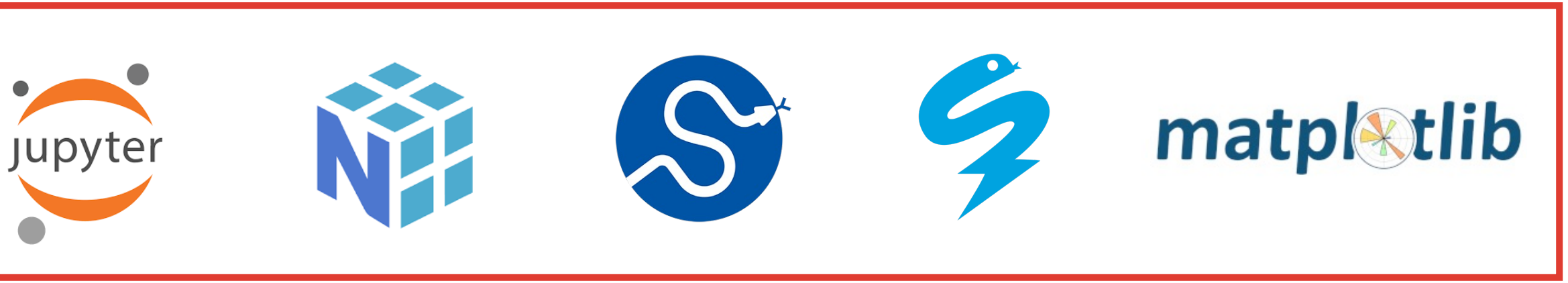
- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren't special enough to break the rules.
- Although practicality beats purity.
- Errors should never pass silently.
- Unless explicitly silenced.
- In the face of ambiguity, refuse the temptation to guess.
- There should be one, and preferably only one, obvious way to do it.
- Although that way may not be obvious at first unless you're Dutch.
- Now is better than never.
- Although never is often better than right now.
- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.
- Namespaces are one honking great idea – let's do more of those!

Python Refresher

- Official Tutorial: https://docs.python.org/3/tutorial/index.html
- freeCodeCamp site and Youtube Channel
- My course: https://github.com/ahmed-f-alrefaie/UCLPythonCourse
- Online version: https://ahmed-f-alrefaie.github.io/UCLPythonCourse/lab/index.html

High Performance Computing with Python

Python HPC tools

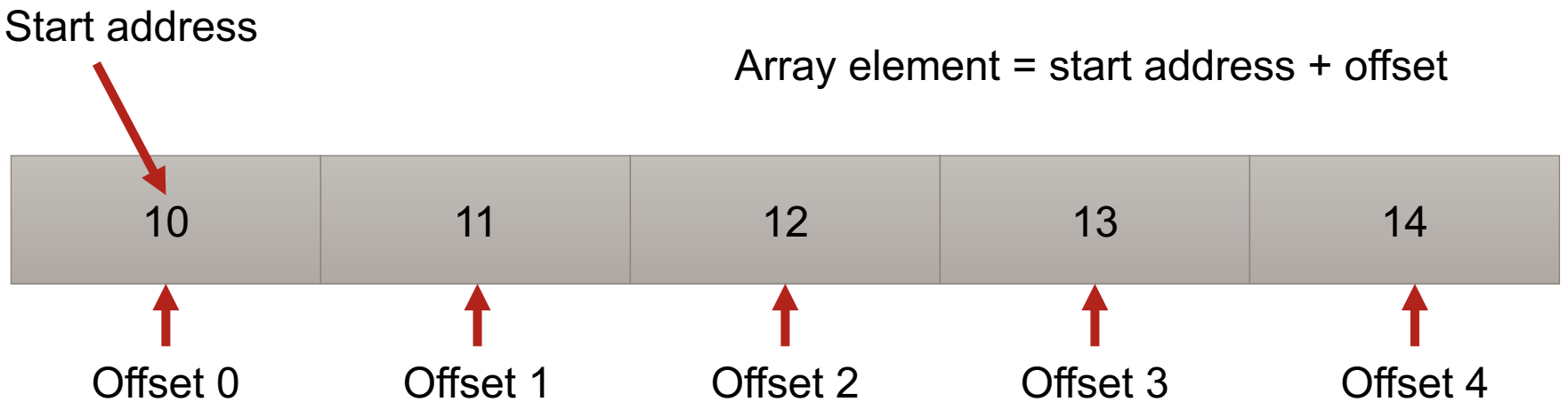


Memory Layout

- Memory can be thought of as a linear collection of consecutive spaces and addresses

Arrays

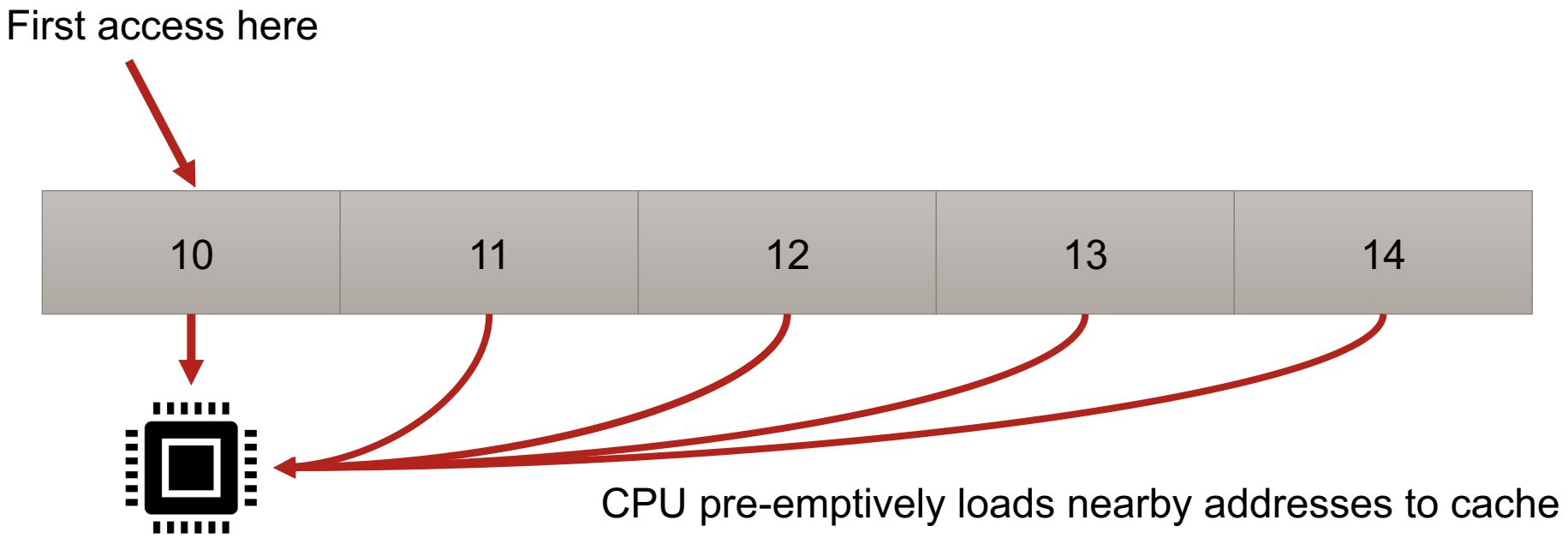
- Arrays are a representation of memory with a starting address, size and offset



- C/C++ uses *malloc* and *new*. Fortran uses *allocate.*
- Offset math is why zero-based indexing is used in most languages.

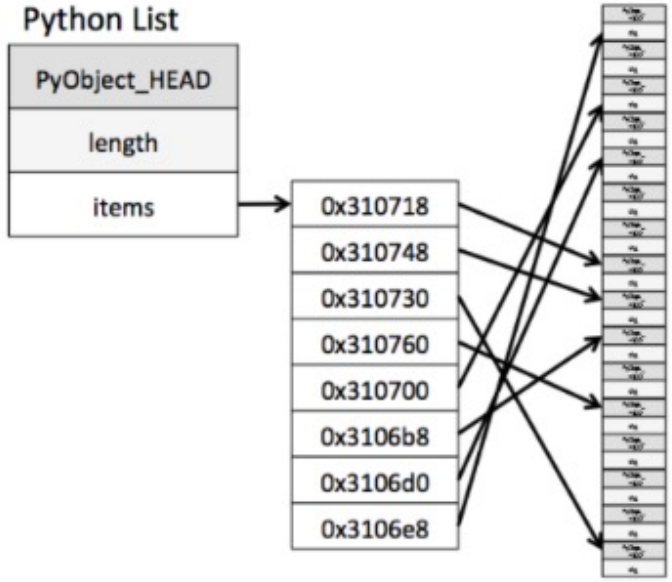
Locality and Cache

- Memory access is slow. Modern CPUs overcome this by caching nearby addresses into fast CPU cache.



Arrays and Python

- Arrays can only store 1 data type in a contiguous manner
  - Array of ints
  - Array of floats
  - Array of strings
- Python lists can store arbitrary data.
  - Array of addresses
  - Data is not guaranteed to be spatially local!
  - CPU can't cache it efficiently!



N-dimensional Arrays

- How are things like matrices stored?

$$A = \begin{bmatrix} 5 & 8 \\ 2 & 7 \end{bmatrix}$$

C-style ordering

- Row based ordering

$$A = \begin{bmatrix} 5 & 8 \\ 2 & 7 \end{bmatrix}$$

$$A[i,j] = A_{addr} + j * N_{row} + i$$

Term is spatially local

Fortran ordering

- Column based ordering

$$A = \begin{bmatrix} 5 & 8 \\ 2 & 7 \end{bmatrix}$$

$$A[i,j] = A_{addr} + j * N_{col} + i$$

Layout and Ordering

- No such thing as an ND array!
- Only 1D represented in a ND manner.
- Consider a libraries preferred layout
- Ignoring data layout leads to inefficiency
  - Lose locality and caching!

BLAS

- *Basic Linear Algebra Subprograms*
- Set of specifications for common linear algebra operations
- Reference implementation in http://www.netlib.org
  - Slow and unoptimized
- Vendors provide optimized versions:
  - Intel MKL
  - ARM Performance Lib
  - nVidia cuBlas
  - AMD rocBlas

BLAS

- BLAS has 3 variants of routines
- BLAS level 1 requires $O(n)$ computational complexity
  - Vector math, dot-products
- BLAS level 2 require $O(n^2)$ computational complexity
  - Matrix-vector operations, triangular solves
- BLAS level 3 require $O(n^3)$ computational complexity
  - Matrix-matrix multiply

LAPACK

- *Linear Algebra Package*
- Built on top of BLAS
- Solving linear equations
- Eigenvalue solver
- Singular Value Decomposition
- Factorisations

L	A	P	A	C	K
L	-A	P	-A	C	-K
L	A	P	A	-C	-K
L	-A	P	-A	-C	K
L	A	-P	-A	C	K
L	-A	-P	A	C	-K

Numpy to the Rescue!

- It is *THE* python package for numerical computing
- True contiguous array datatype
- Provides both layouts
  - C-style default
- Fast data types for vectors and matrices
- Optimized math operations
- Linear algebra operations
  - Uses BLAS and LAPACK

Numpy Demo

Parallel Computing

- Modern computing is highly parallel.
- Modern CPUs have parallel execution of operations and multiple cores
- Other computing hardware like GPUs are extremely parallel.
- HPC clusters also allow parallel execution across machines.

SIMD Acceleration

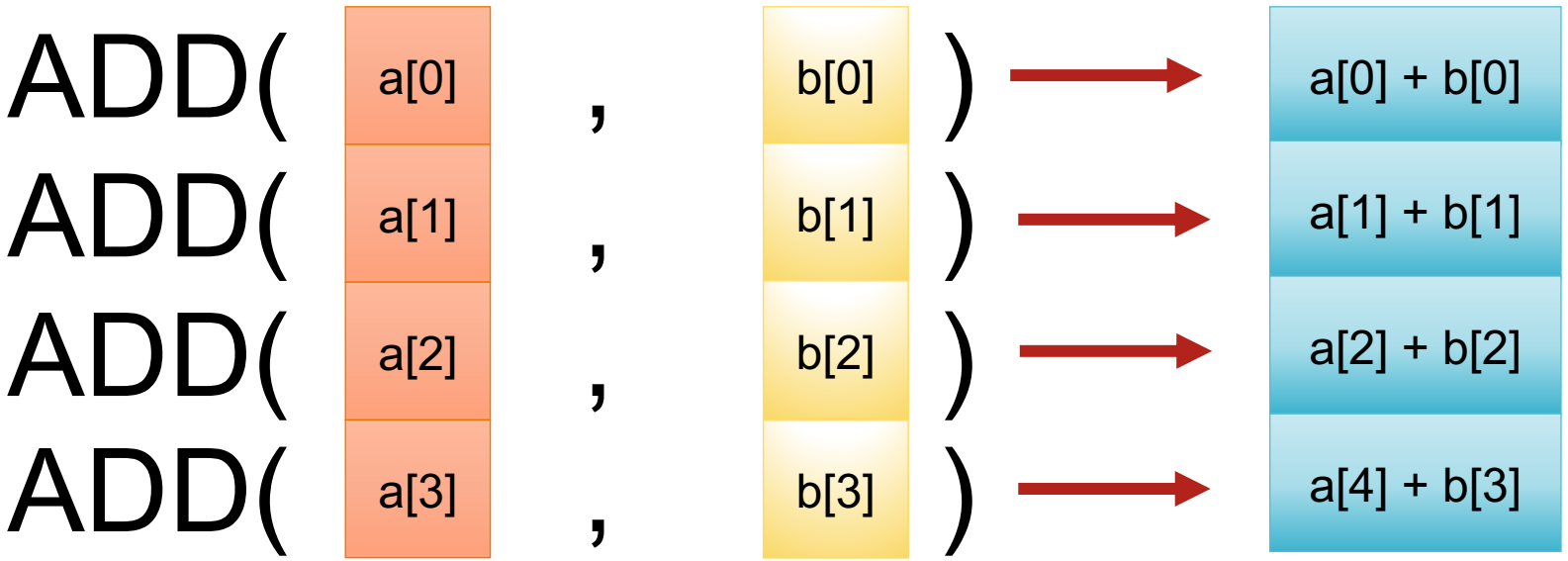
- *Single Instruction Multiple Data*
- Perform same operation *once* on multiple datasets
- A set of *vector registers* and *instructions*
- *Advanced Vector Extensions* (AVX) is an example
- CPUs must support this to use it.

SIMD Acceleration

- *Vector registers* are wide internal CPU storage.
  - AVX allows for 256-bit wide vector registers
  - 8 floats or 4 doubles

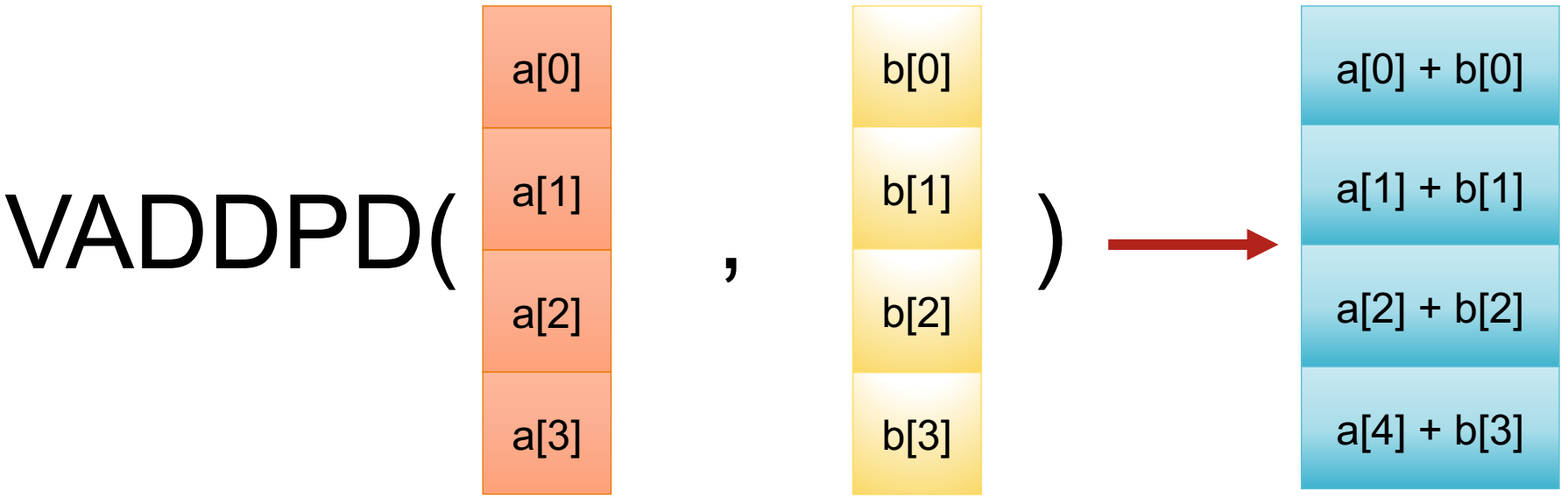
- *Vector Instructions* sets are operations on these vector units
  - VADDPD – **V**ector **add** two values '**p**acked' **d**ouble precision
  - VSQRTPD – **V**ector **sq**uare **r**oo**t** '**p**acked' double precision

SIMD ADD example



4 instructions required!!

SIMD ADD example



1 instruction only!
4x throughput!

SIMD Acceleration

- Low level tool
- Applied during compilation of code
- Can’t invoke it directly in Python
- Several python libraries allow exploiting SIMD
  - Numpy: SIMD if underlying BLAS is compiled with it.
  - Numexpr: compiles array operations into fast SIMD instructions
  - Numba: Just-in-Time compiler that can compile python code into SIMD accelerated code
- The last two we will explore later!

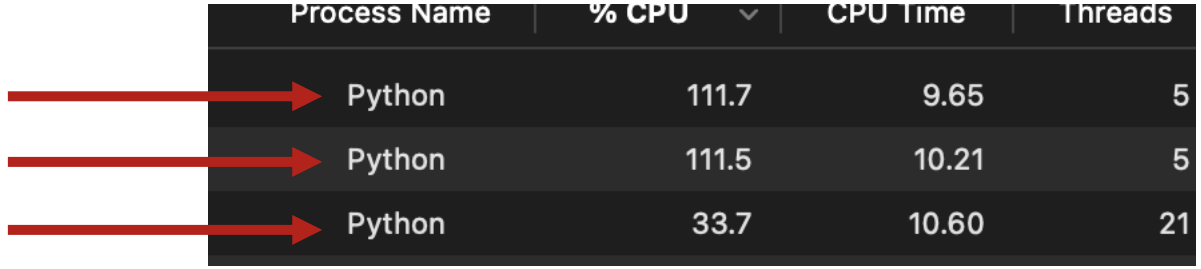
Multi-threading parallel

- SIMD is *data-level parallelism*
  - Only executed in a single core
- Code can be executed over multiple cores
- A performant application should exploit multiple cores
- Two execution units are *process* and *threads*

Process

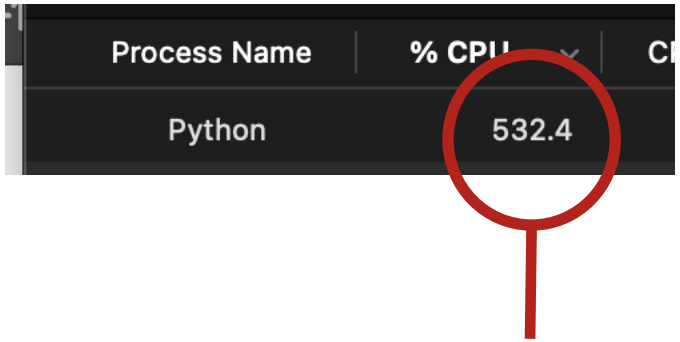
- Self contained execution of code with memory
- Strictly separated
- Data exchange via operating system or other architecture
  - *Message Passing Interface* (MPI) for example

Separate Python processes



Thread

- An execution stream within a process
- Memory shared and freely accessed between threads



5 cores in a single process via threading

Threading Demo

The GIL

- *Global Interpreter Lock*
- Python ‘feature’
- Only one Python thread can execute at a time
- Emerged from legacy decisions
- CPython codebase easier to manage and improves performance
- Effort to remove it
  - Performance drops significantly
  - Everything breaks


GIL
Multithreading
cpu bound code

Whats the point of threading?

- Some tasks need to wait for something
  - Reading a file
  - Waiting for network
- *I/O bound*
- Another thread can run while the other is waiting.
  - ‘Releasing the GIL’

Multi-process execution

Python has a solution!

If you can’t release the GIL!

Make another!

Multi-process execution

- *multiprocessing* module
- Spawn any number of Python processes
- True ‘multicore’ execution
- Data-movement slow
  - Requires ‘pickling’ data and piping it
  - Terrible for big arrays

Multiprocessing Demo

Parallel processing

- There are many options to handle parallelism
- Pros and Cons of each method
- You can blend both parallel methods as well
  - Multiprocessing with threads
- Depends on what you want to achieve
- We will explore ways to overcome this in the coming weeks

Next week

- Next Friday: Using Numba and numexpr to exploit SIMD and parallelism

- Monday: Hands-on with numpy.